

Actividad 2: Lógica de Programación y Programación Orientada a Objetos

Daniel Libardo Cardenas Valbuena
dcardenasva@unal.edu.co

Docente: Walter Hugo Arboleda Mazo

Facultad de Minas
Universidad Nacional de Colombia Sede Medellin
8 de Octubre, 2026

Tabla de contenido

Estructura de Proyecto *3*

Ejercicio 2.1 – Personas *4*

Ejercicio 2.2 – Planeta *5*

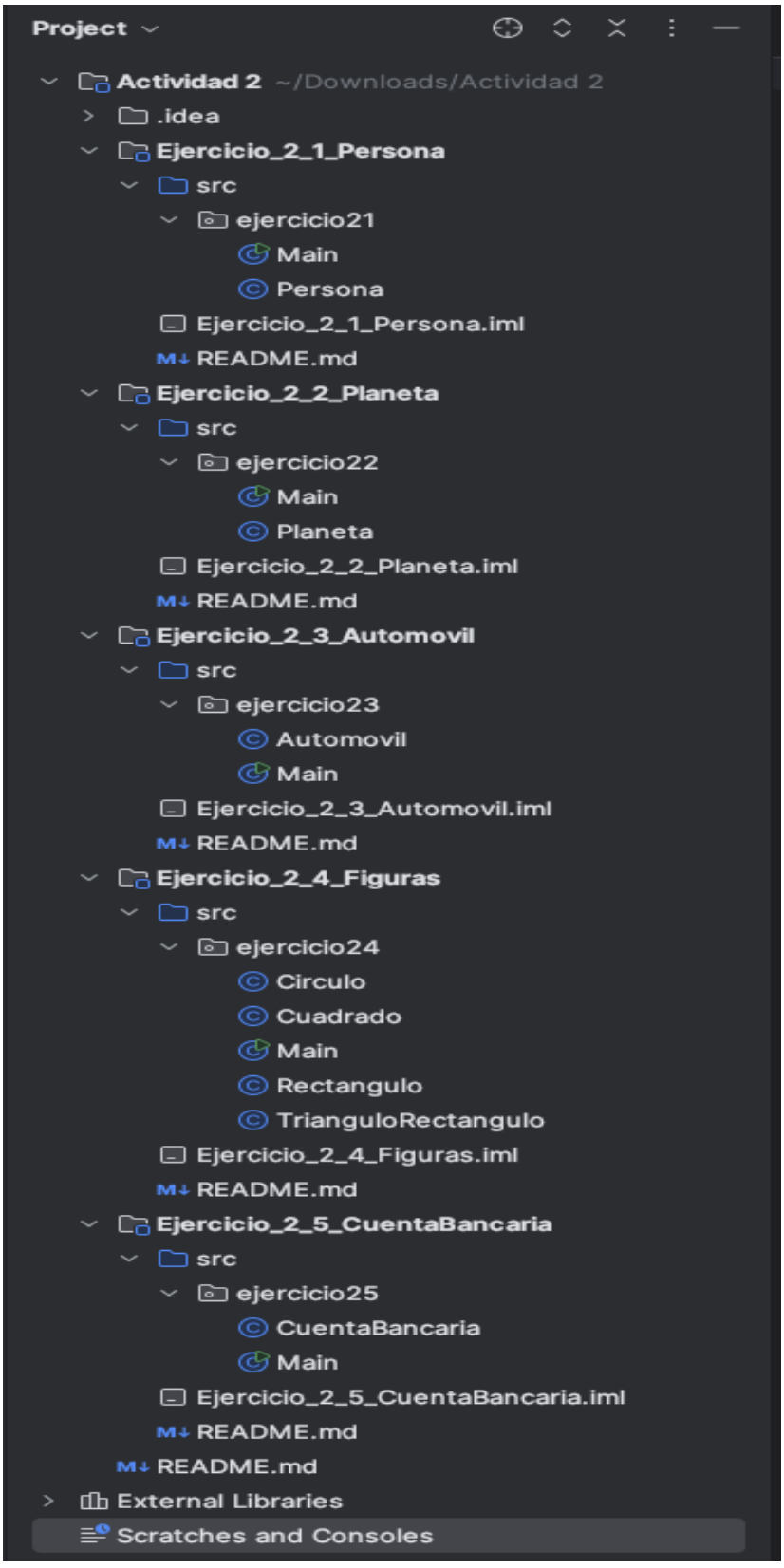
Ejercicio 2.3 – Automóvil *8*

Ejercicio 2.4 – Figuras *12*

Ejercicio 2.5 – Cuenta Bancaria *14*

Link del repositorio *16*

Estructura de Proyecto



Ejercicio 2.1 – Personas

Main.java

```java
package ejercicio21;

public class Main {
    public static void main(String[] args) {
        Persona persona1 = new Persona("Pedro", "Pérez", "1053121010",
1998);
        Persona persona2 = new Persona("Luis", "León", "1053223344",
2001);
        persona1.imprimir();
        persona2.imprimir();
    }
}
```

Persona.java

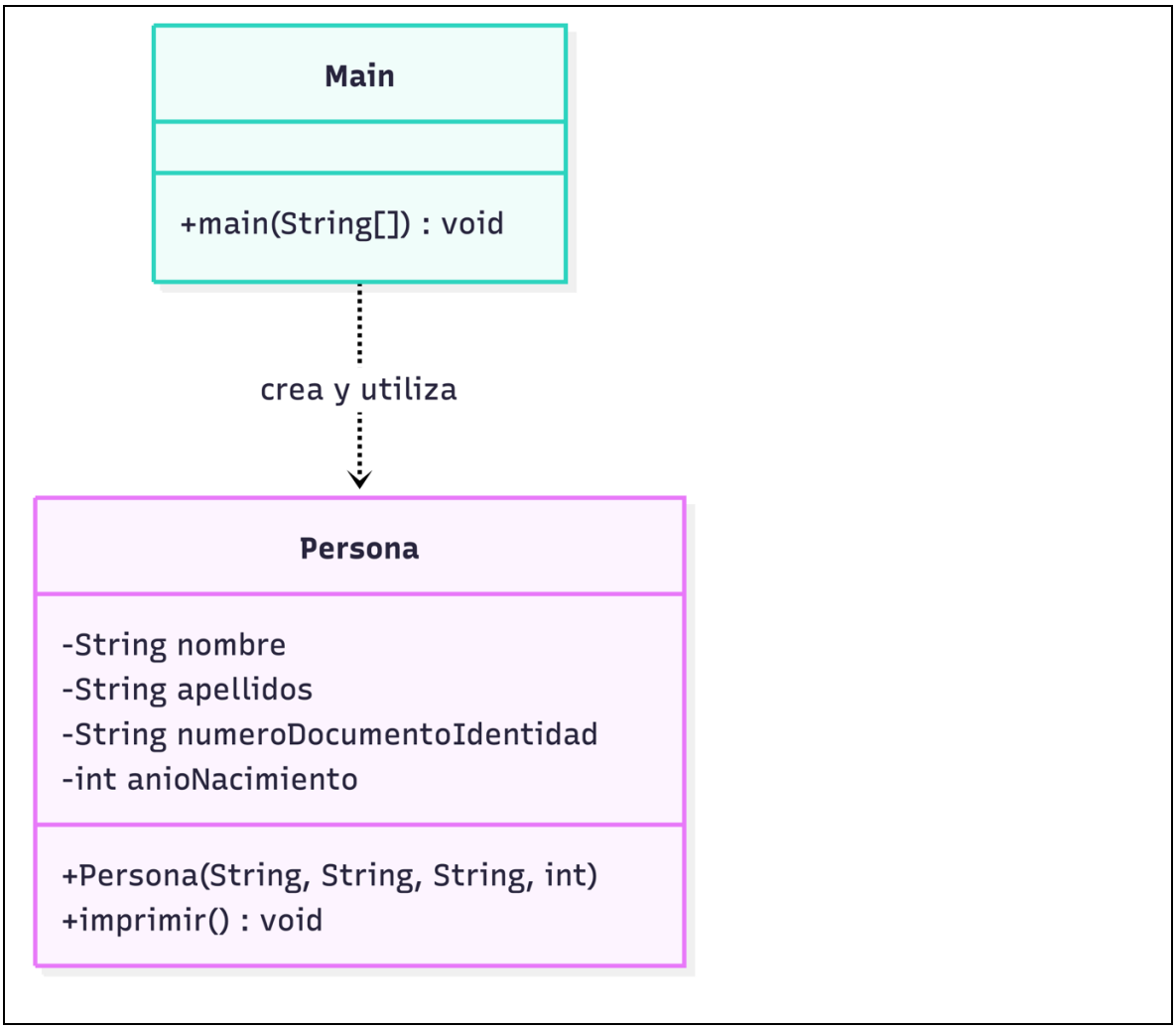
```java
package ejercicio21;

/** Representa una persona y sus datos básicos. */
public class Persona {
    private final String nombre;
    private final String apellidos;
    private final String numeroDocumentoIdentidad;
    private final int anioNacimiento;

    public Persona(String nombre, String apellidos, String
numeroDocumentoIdentidad, int anioNacimiento) {
        if (nombre == null || nombre.isBlank()) throw new
IllegalArgumentException("El nombre es obligatorio.");
        if (apellidos == null || apellidos.isBlank()) throw new
IllegalArgumentException("Los apellidos son obligatorios.");
        if (numeroDocumentoIdentidad == null ||
numeroDocumentoIdentidad.isBlank()) throw new
IllegalArgumentException("El documento es obligatorio.");
        if (anioNacimiento <= 0) throw new IllegalArgumentException("El
año de nacimiento debe ser válido.");
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.numeroDocumentoIdentidad = numeroDocumentoIdentidad;
        this.anioNacimiento = anioNacimiento;
    }

    public void imprimir() {
        System.out.println("Nombre: " + nombre);
        System.out.println("Apellidos: " + apellidos);
        System.out.println("Documento de identidad: " +
numeroDocumentoIdentidad);
        System.out.println("Año de nacimiento: " + anioNacimiento);
        System.out.println();
    }
}
```

Diagrama de ULM:



Ejercicio 2.2 – Planeta

Main.java

```java
package ejercicio22;

public class Main {
    public static void main(String[] args) {
        Planeta planeta = new Planeta("Júpiter", 95, 1.898e27,
1.4313e15, 139820,
                778, Planeta.TipoPlaneta.GASEOSO, true);
        planeta.imprimir();
        System.out.printf("Densidad: %.4e kg/km³%n",
planeta.calcularDensidad());
        System.out.println("¿Es planeta exterior?: " +
(planeta.esPlanetaExterior() ? "Sí" : "No"));
    }
}
```

Planeta.java

```
package ejercicio22;


public class Planeta {
    public enum TipoPlaneta { GASEOSO, TERRESTRE, ENANO }

    private final String nombre;
    private final int cantidadSatelites;
    private final double masa;       // kg
    private final double volumen;    // km³
    private final int diametro;      // km
    private final int distanciaSol;  // millones de km
    private final TipoPlaneta tipo;
    private final boolean observable;

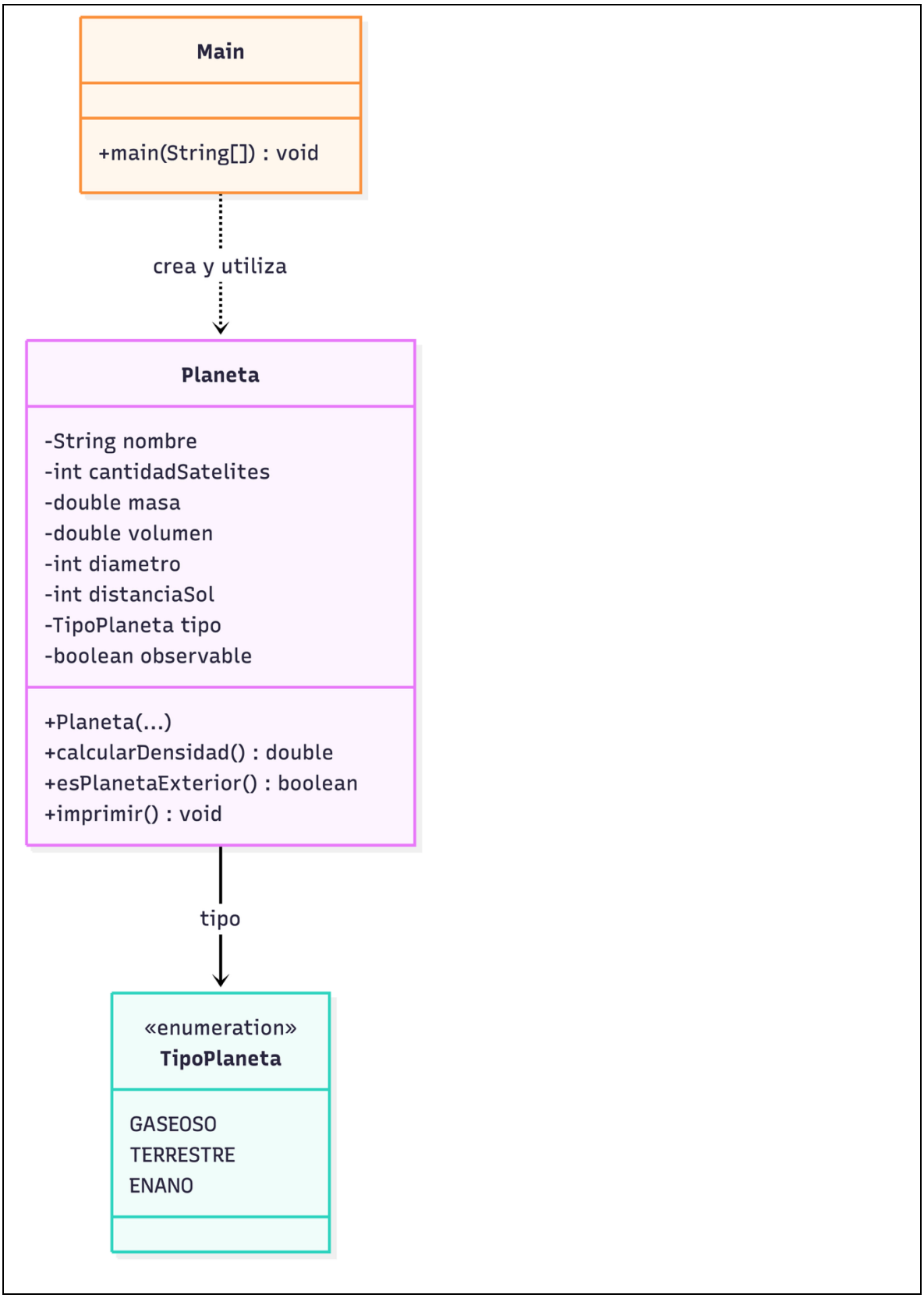
    public Planeta(String nombre, int cantidadSatelites, double masa,
double volumen,
                   int diametro, int distanciaSol, TipoPlaneta tipo,
boolean observable) {
        if (nombre == null || nombre.isBlank()) throw new
IllegalArgumentException("El nombre es obligatorio.");
        if (cantidadSatelites < 0 || masa <= 0 || volumen <= 0 ||
diametro <= 0 || distanciaSol < 0)
            throw new IllegalArgumentException("Los datos numéricos del
planeta no son válidos.");
        if (tipo == null) throw new IllegalArgumentException("Debe
especificarse el tipo de planeta.");
        this.nombre = nombre; this.cantidadSatelites =
cantidadSatelites; this.masa = masa;
        this.volumen = volumen; this.diametro = diametro;
this.distanciaSol = distanciaSol;
        this.tipo = tipo; this.observable = observable;
    }

    public double calcularDdensidad() { return masa / volumen; }


    public boolean esPlanetaExterior() { return distanciaSol > 227.9; }

    public void imprimir() {
        System.out.println("Planeta: " + nombre);
        System.out.println("Satélites: " + cantidadSatelites);
        System.out.println("Masa (kg): " + masa);
        System.out.println("Volumen (km³): " + volumen);
        System.out.println("Diámetro (km): " + diametro);
        System.out.println("Distancia al Sol (millones de km): " +
distanciaSol);
        System.out.println("Tipo: " + tipo);
        System.out.println("Observable: " + (observable ? "Sí" :
"No"));
    }
}
```

Diagrama de ULM:


Main
+main(String[]) : void
crea y utiliza
Planeta
-String nombre
-int cantidadSatelites
-double masa
-double volumen
-int diametro
-int distanciaSol
-TipoPlaneta tipo
-boolean observable
+Planeta(...)
+calcularDensidad() : double
+esPlanetaExterior() : boolean
+imprimir() : void
tipo
«enumeration»
TipoPlaneta
GASEOSO
TERRESTRE
ENANO

Ejercicio 2.3 – Automóvil

Main.java

```
package ejercicio23;

public class Main {
    public static void main(String[] args) {
        Automovil auto = new Automovil("Toyota", 2022, 2.0,
                Automovil.TipoCombustible.GASOLINA,
Automovil.TipoAutomovil.SUV,
                5, 5, 180, Automovil.Color.AZUL, 60);
        auto.imprimir();
        auto.acelerar(30);
        System.out.println("Después de acelerar: " +
auto.getVelocidadActual() + " km/h");
        auto.desacelerar(20);
        System.out.println("Después de desacelerar: " +
auto.getVelocidadActual() + " km/h");
        System.out.printf("Tiempo para recorrer 100 km: %.2f horas%n",
auto.calcularTiempoLlegada(100));
        auto.frenar();
        System.out.println("Después de frenar: " +
auto.getVelocidadActual() + " km/h");
    }
}
```

Automovil.java

```
package ejercicio23;

public class Automovil {
    public enum TipoCombustible { GASOLINA, BIOETANOL, DIESEL,
BIODIESEL, GAS_NATURAL }
    public enum TipoAutomovil { CIUDAD, SUBCOMPACTO, COMPACTO,
FAMILIAR, EJECUTIVO, SUV }
    public enum Color { BLANCO, NEGRO, ROJO, NARANJA, AMARILLO, VERDE,
AZUL, VIOLETA }

    private String marca;
    private int modelo;
    private double motor;
    private TipoCombustible tipoCombustible;
    private TipoAutomovil tipoAutomovil;
    private int numeroPuertas;
    private int cantidadAsientos;
    private int velocidadMaxima;
    private Color color;
    private int velocidadActual;

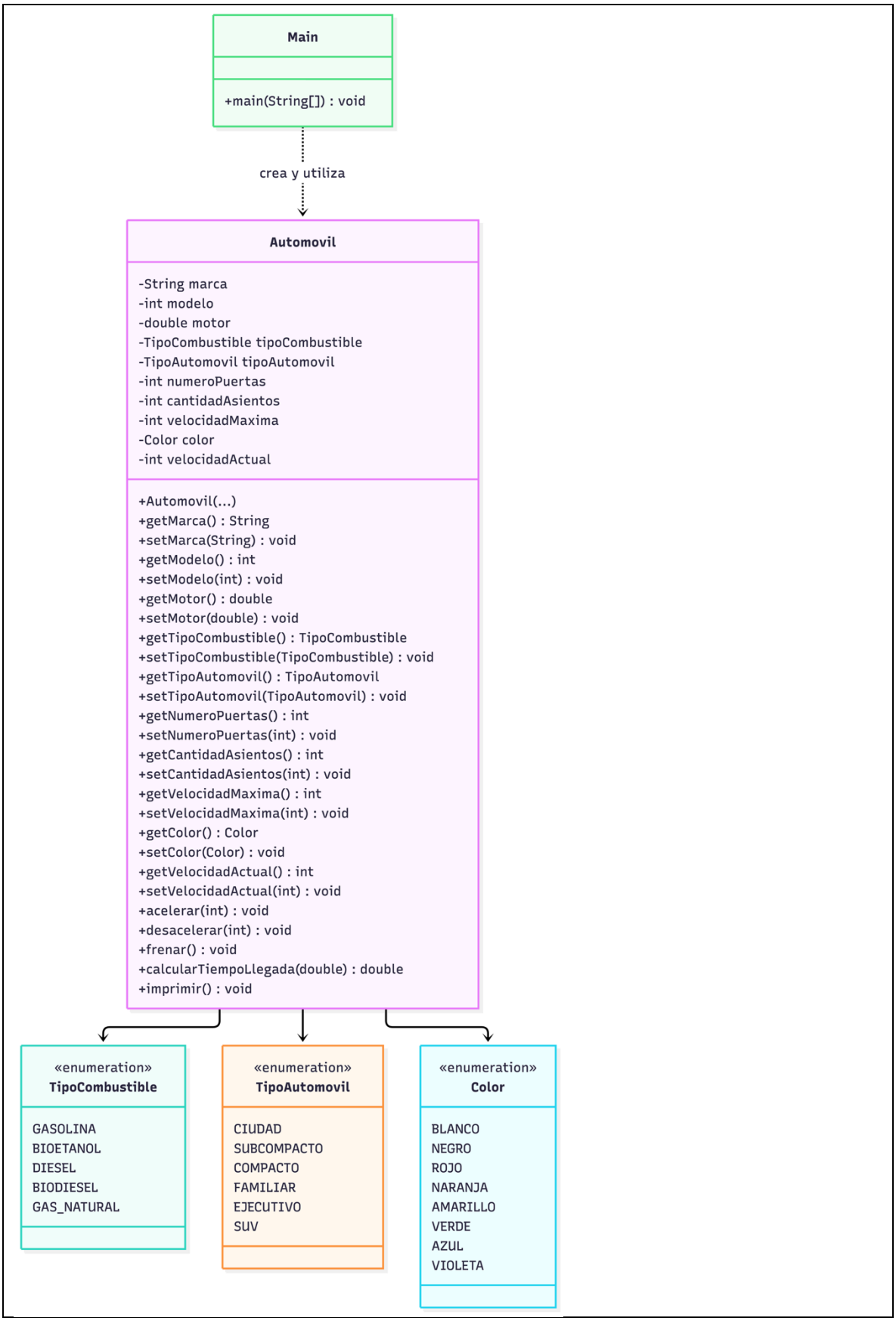
    public Automovil(String marca, int modelo, double motor,
TipoCombustible combustible,
                     TipoAutomovil tipo, int puertas, int asientos, int
velocidadMaxima,
                     Color color, int velocidadActual) {
```

```
        setMarca(marca); setModelo(modelo); setMotor(motor);
setTipoCombustible(combustible);
        setTipoAutomovil(tipo); setNumeroPuertas(puertas);
setCantidadAsientos(asientos);
        setVelocidadMaxima(velocidadMaxima); setColor(color);
setVelocidadActual(velocidadActual);
    }
    public String getMarca() { return marca; }
    public void setMarca(String v) { if (v == null || v.isBlank())
throw new IllegalArgumentException("Marca obligatoria."); marca=v; }
    public int getModelo() { return modelo; }
    public void setModelo(int v) { if (v < 1886) throw new
IllegalArgumentException("Modelo no válido."); modelo=v; }
    public double getMotor() { return motor; }
    public void setMotor(double v) { if (v <= 0) throw new
IllegalArgumentException("El cilindraje debe ser positivo."); motor=v;
}
    public TipoCombustible getTipoCombustible() { return
tipoCombustible; }
    public void setTipoCombustible(TipoCombustible v) { if (v == null)
throw new IllegalArgumentException("Combustible obligatorio.");
tipoCombustible=v; }
    public TipoAutomovil getTipoAutomovil() { return tipoAutomovil; }
    public void setTipoAutomovil(TipoAutomovil v) { if (v == null)
throw new IllegalArgumentException("Tipo obligatorio.");
tipoAutomovil=v; }
    public int getNumeroPuertas() { return numeroPuertas; }
    public void setNumeroPuertas(int v) { if (v < 1) throw new
IllegalArgumentException("Número de puertas no válido.");
numeroPuertas=v; }
    public int getCantidadAsientos() { return cantidadAsientos; }
    public void setCantidadAsientos(int v) { if (v < 1) throw new
IllegalArgumentException("Cantidad de asientos no válida.");
cantidadAsientos=v; }
    public int getVelocidadMaxima() { return velocidadMaxima; }
    public void setVelocidadMaxima(int v) { if (v < 1) throw new
IllegalArgumentException("Velocidad máxima no válida.");
velocidadMaxima=v; if (velocidadActual > v) velocidadActual=v; }
    public Color getColor() { return color; }
    public void setColor(Color v) { if (v == null) throw new
IllegalArgumentException("Color obligatorio."); color=v; }
    public int getVelocidadActual() { return velocidadActual; }
    public void setVelocidadActual(int v) { if (v < 0 || v >
velocidadMaxima) throw new IllegalArgumentException("La velocidad debe
estar entre 0 y la máxima."); velocidadActual=v; }

    public void acelerar(int incremento) { if (incremento < 0) throw
new IllegalArgumentException("Incremento negativo.");
velocidadActual=Math.min(velocidadMaxima, velocidadActual+incremento);
}
    public void desacelerar(int decremento) { if (decremento < 0) throw
new IllegalArgumentException("Decremento negativo.");
velocidadActual=Math.max(0, velocidadActual-decremento); }
    public void frenar() { velocidadActual=0; }
    public double calcularTiempoLlegada(double distanciaKm) {
        if (distanciaKm < 0) throw new IllegalArgumentException("La
distancia no puede ser negativa.");
```

```
        if (velocidadActual == 0) throw new IllegalStateException("El
automóvil debe estar en movimiento.");
        return distanciaKm / velocidadActual;
    }
    public void imprimir() {
        System.out.println("Marca: " + marca + " | Modelo: " + modelo +
" | Motor: " + motor + " L");
        System.out.println("Combustible: " + tipoCombustible + " |
Tipo: " + tipoAutomovil + " | Color: " + color);
        System.out.println("Puertas: " + numeroPuertas + " | Asientos:
" + cantidadAsientos);
        System.out.println("Velocidad máxima: " + velocidadMaxima + "
km/h | Velocidad actual: " + velocidadActual + " km/h");
    }
}
```

Diagrama de ULM:


Main
+main(String[]) : void
crea y utiliza
Automovil
-String marca
-int modelo
-double motor
-TipoCombustible tipoCombustible
-TipoAutomovil tipoAutomovil
-int numeroPuertas
-int cantidadAsientos
-int velocidadMaxima
-Color color
-int velocidadActual
+Automovil(...)
+getMarca() : String
+setMarca(String) : void
+getModelo() : int
+setModelo(int) : void
+getMotor() : double
+setMotor(double) : void
+getTipoCombustible() : TipoCombustible
+setTipoCombustible(TipoCombustible) : void
+getTipoAutomovil() : TipoAutomovil
+setTipoAutomovil(TipoAutomovil) : void
+getNumeroPuertas() : int
+setNumeroPuertas(int) : void
+getCantidadAsientos() : int
+setCantidadAsientos(int) : void
+getVelocidadMaxima() : int
+setVelocidadMaxima(int) : void
+getColor() : Color
+setColor(Color) : void
+getVelocidadActual() : int
+setVelocidadActual(int) : void
+acelerar(int) : void
+desacelerar(int) : void
+frenar() : void
+calcularTiempoLlegada(double) : double
+imprimir() : void
«enumeration»
TipoCombustible
GASOLINA
BIOETANOL
DIESEL
BIODIESEL
GAS_NATURAL
«enumeration»
TipoAutomovil
CIUDAD
SUBCOMPACTO
COMPACTO
FAMILIAR
EJECUTIVO
SUV
«enumeration»
Color
BLANCO
NEGRO
ROJO
NARANJA
AMARILLO
VERDE
AZUL
VIOLETA

Ejercicio 2.4 – Figuras

Main.java

```
package ejercicio24;

public class Main {
    public static void main(String[] args) {
        Circulo circulo = new Circulo(3.0);
        Rectangulo rectangulo = new Rectangulo(5.0, 2.0);
        Cuadrado cuadrado = new Cuadrado(4.0);
        TrianguloRectangulo triangulo = new TrianguloRectangulo(3.0,
4.0);
        System.out.printf("Círculo — área: %.2f cm², perímetro: %.2f
cm%n", circulo.calcularArea(), circulo.calcularPerimetro());
        System.out.printf("Rectángulo — área: %.2f cm², perímetro: %.2f
cm%n", rectangulo.calcularArea(), rectangulo.calcularPerimetro());
        System.out.printf("Cuadrado — área: %.2f cm², perímetro: %.2f
cm%n", cuadrado.calcularArea(), cuadrado.calcularPerimetro());
        System.out.printf("Triángulo rectángulo — hipotenusa: %.2f cm,
área: %.2f cm², perímetro: %.2f cm, clasificación: %s%n",
                triangulo.calcularHipotenusa(),
triangulo.calcularArea(), triangulo.calcularPerimetro(),
triangulo.determinarTipo());
    }
}
```

Circulo.java

```
package ejercicio24;

public class Circulo {
    private final double radio;
    public Circulo(double radio) { if (radio <= 0) throw new
IllegalArgumentException("El radio debe ser positivo.");
this.radio=radio; }
    public double calcularArea() { return Math.PI * radio * radio; }
    public double calcularPerimetro() { return 2 * Math.PI * radio; }
}
```

Cuadrado.java

```
package ejercicio24;

public class Cuadrado extends Rectangulo {
    public Cuadrado(double lado) { super(lado, lado); }
}
```

Rectangulo.java

```
package ejercicio24;

public class Rectangulo {
    protected final double base, altura;
    public Rectangulo(double base, double altura) { if (base <= 0 ||
altura <= 0) throw new IllegalArgumentException("Base y altura deben
ser positivas."); this.base=base; this.altura=altura; }
    public double calcularArea() { return base * altura; }
```

```
    public double calcularPerimetro() { return 2 * (base + altura); }
}
```

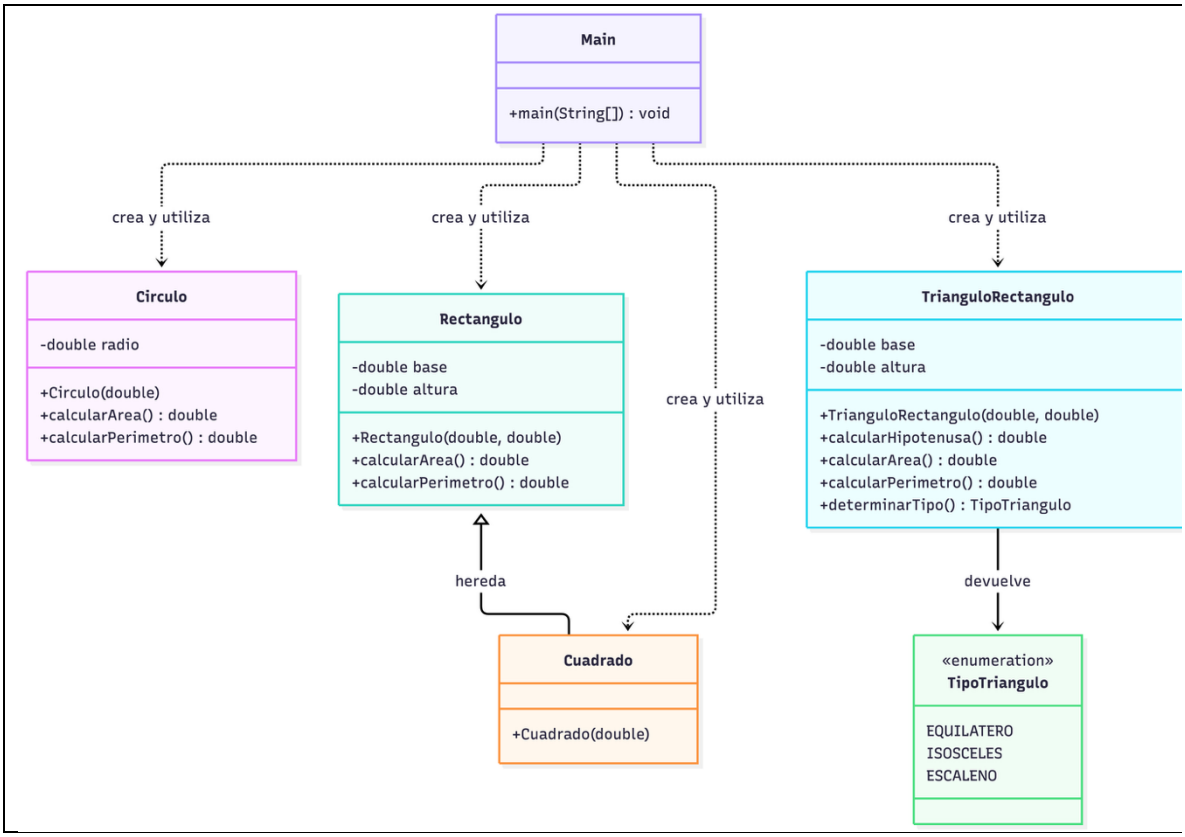
TrianguloRectangulo.java

```
package ejercicio24;

public class TrianguloRectangulo {
    public enum TipoTriangulo { EQUILATERO, ISOSCELES, ESCALENO }
    private final double base, altura;
    public TrianguloRectangulo(double base, double altura) { if (base
<= 0 || altura <= 0) throw new IllegalArgumentException("Base y altura
deben ser positivas."); this.base=base; this.altura=altura; }
    public double calcularHipotenusa() { return Math.hypot(base,
altura); }
    public double calcularArea() { return base * altura / 2.0; }
    public double calcularPerimetro() { return base + altura +
calcularHipotenusa(); }


    public TipoTriangulo determinarTipo() {
        double[] lados = {base, altura, calcularHipotenusa()};
        int iguales = 0;
        if (casiIgual(lados[0], lados[1])) iguales++;
        if (casiIgual(lados[0], lados[2])) iguales++;
        if (casiIgual(lados[1], lados[2])) iguales++;
        if (iguales == 3) return TipoTriangulo.EQUILATERO;
        if (iguales > 0) return TipoTriangulo.ISOSCELES;
        return TipoTriangulo.ESCALENO;
    }
    private boolean casiIgual(double a, double b) { return Math.abs(a-
b) < 1e-9; }
}
```

Diagrama de ULM:



Ejercicio 2.5 – Cuenta Bancaria

Main.java

```java
package ejercicio25;

public class Main {
    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria("Ana María", "Gómez
Pérez", "00123456789", CuentaBancaria.TipoCuenta.AHORROS);
        cuenta.imprimir();
        System.out.println("Consignación de $250000: " +
(cuenta.consignar(250000) ? "exitosa" : "rechazada"));
        System.out.println("Consignación de $50000: " +
(cuenta.consignar(50000) ? "exitosa" : "rechazada"));
        System.out.printf("Saldo consultado: $%,.2f%n",
cuenta.consultarSaldo());
        System.out.println("Retiro de $100000: " +
(cuenta.retirar(100000) ? "exitoso" : "rechazado"));
        System.out.println("Retiro de $500000: " +
(cuenta.retirar(500000) ? "exitoso" : "rechazado por saldo
insuficiente"));
        cuenta.imprimir();
    }
}
```

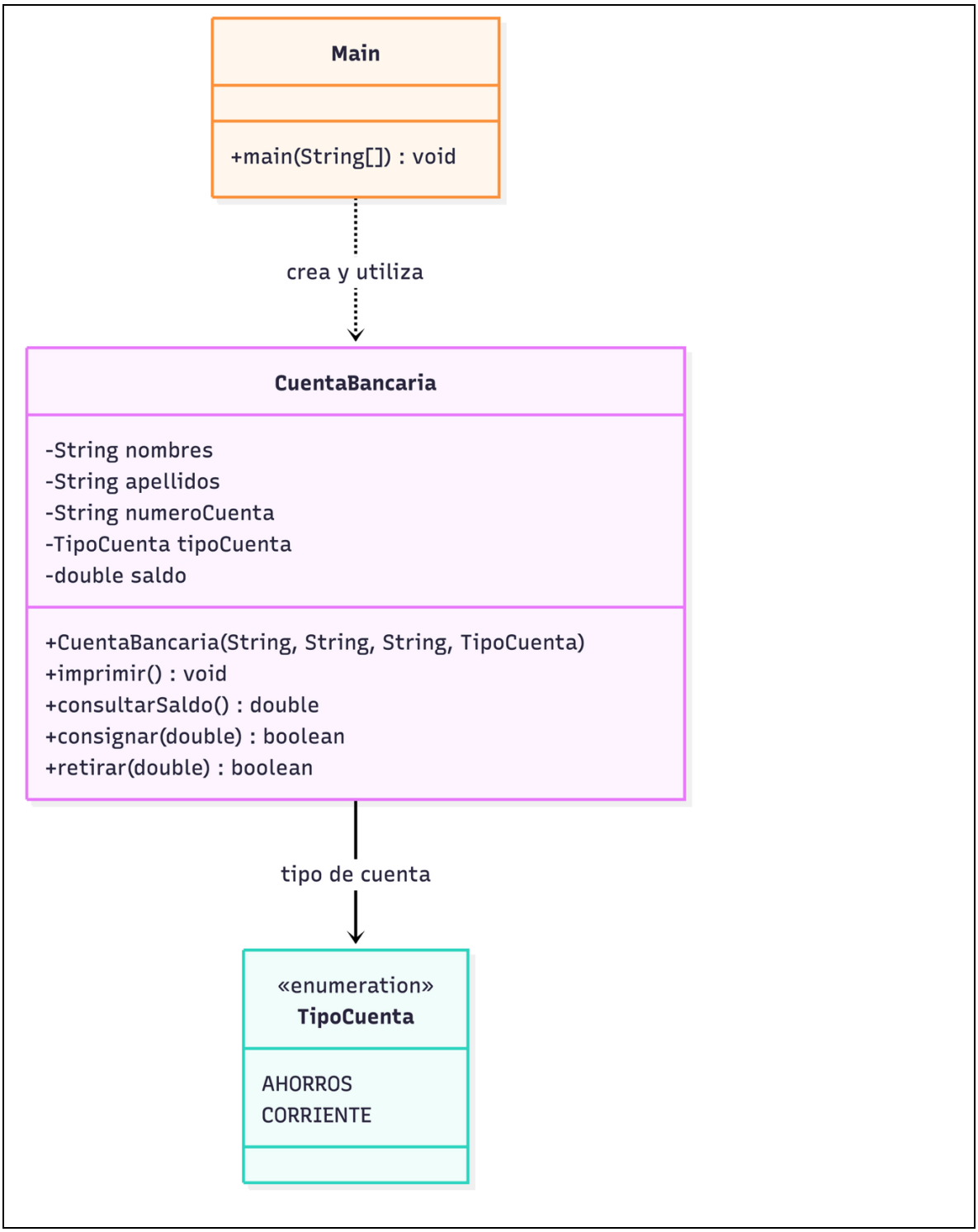
CuentaBancaria.java

```java
package ejercicio25;


public class CuentaBancaria {
    public enum TipoCuenta { AHORROS, CORRIENTE }
    private final String nombres;
    private final String apellidos;
    private final String numeroCuenta;
    private final TipoCuenta tipoCuenta;
    private double saldo;

    public CuentaBancaria(String nombres, String apellidos, String
numeroCuenta, TipoCuenta tipoCuenta) {
        if (nombres == null || nombres.isBlank() || apellidos == null
|| apellidos.isBlank() || numeroCuenta == null ||
numeroCuenta.isBlank())
            throw new IllegalArgumentException("Nombres, apellidos y
número de cuenta son obligatorios.");
        if (tipoCuenta == null) throw new IllegalArgumentException("El
tipo de cuenta es obligatorio.");
        this.nombres=nombres; this.apellidos=apellidos;
this.numeroCuenta=numeroCuenta; this.tipoCuenta=tipoCuenta;
this.saldo=0.0;
    }
    public void imprimir() {
        System.out.println("Titular: " + nombres + " " + apellidos);
        System.out.println("Número de cuenta: " + numeroCuenta);
        System.out.println("Tipo de cuenta: " + tipoCuenta);
        System.out.printf("Saldo: $%,.2f%n", saldo);
    }
    public double consultarSaldo() { return saldo; }
    public boolean consignar(double valor) {
        if (valor <= 0 || !Double.isFinite(valor)) return false;
        saldo += valor; return true;
    }
    public boolean retirar(double valor) {
        if (valor <= 0 || !Double.isFinite(valor) || valor > saldo)
return false;
        saldo -= valor; return true;
    }
}
```

Diagrama de ULM:



Link del repositorio

https://github.com/Danniel-C07/Actividad_2---POO.git